

JT_DEV_B42_1.6

React todo list

1 Initial situation

- 1.1 On your last todo list project, the application was developed with a really basic architecture; An html web page, a web API, and some TypeScript functions to handle app interactivity. Nowadays, using vanilla JavaScript or TypeScript to create web applications is not a common practice (eventually for small applications). Most of the time, web frameworks or libraries are used to create web applications. They provide a structure to your codebase, simplifies some tasks and provide reusability of code.
- 1.2 There are many popular web frameworks, such as [Angular](#), [Vue](#), [Svelte](#) or [Solid](#). In 2026, the most popular one is [React](#). React is a library created by Facebook in 2013. Used by many big companies such as Facebook, Instagram, Netflix, Airbnb and smaller ones. React is based on components, which are reusable pieces of code that represent a part of the user interface (like functions but for UI). Components can be nested, managed and handled independently, which allows for complex applications to be built from simple building blocks.
- 1.3 As for programming languages, there is no « best » framework, each one has its own advantages and disadvantages. But React is a good choice for this project because it is widely used, has a large community and a lot of resources available online. Moreover, React is often used with TypeScript, which you already know. This will offer you a good starting point with web frameworks.
- 1.4 The goal of this project is to help you understand and apply modern front-end development practices by rebuilding your existing to-do list application using React. Through this exercise, you will learn how to structure a React application, interact with a RESTful API, and implement reusable components.

2 Skills from the ICT training plan worked on

Skills ranked by importance of implementation in the project: [consult the training plan](#).

- 2.1 g5 : Implémenter les applications et les interfaces selon le concept en respectant les exigences de sécurité.
- 2.2 a3 : Rechercher des informations sur des solutions ICT et sur les innovations.
- 2.3 a6 : Vérifier l'avancement des projets ICT et des tâches et en faire état selon le modèle de procédure.
- 2.4 a4 : Planifier les projets ICT et les tâches selon un modèle de procédure.
- 2.5 g6 : Vérifier la qualité et la sécurité des applications et des interfaces.
- 2.6 g4 : Elaborer des variantes de mise en œuvre d'une application et développer la solution sur le plan conceptuel.
- 2.7 h3 : Implémenter le processus de livraison des applications.

3 Deliverables

- 3.1 Github repository containing your project, it must work out of the box (following the setup guide provided in your README).
- 3.2 A working Github pages deployment of your project.
- 3.3 Your GitHub project board must be completed (all tasks closed by a [close keyword](#)).

4 Andragogic conditions

- 4.1 Work is carried out independently, on an individual basis



- 4.2 You are allowed to help each other, share ideas and discuss solutions, but each trainee must provide their own implementation with their own code.
- 4.3 Apprentices have access to all the resources they need, particularly on the web.
- 4.4 Don't hesitate to ask AI for explanations, but avoid asking it for answers. The more you ask for answers, the less you learn.
- 4.5 **Maximum allowed time for the project: 10 full days (7.5 hours per day).** You must manage your time independently. On project beginning, inform your trainer of the start date and time.

5 Steps

- 5.1 Create a new GitHub Project from [the react todo template](#) to your account (Use the «...» then «Make a copy» menu). Select your account to host the Project on and name the project with your GitHub pseudo, for example `react-todo-<your-username>`. **Do not forget to tick the «Draft issues will be copied if selected» checkbox!**
- 5.2 Then, you just have to implement all the features requested in the GitHub Project. **For each task, take time to read them attentively and complete the `estimate` field with the number of hours you expect to work on it. Before merging your code check that all acceptance criteria are met, take review feedback into account. Then, complete the `final` field with the actual time spent on the task.**

6 Resources

- 6.1 <https://react.dev/learn>
- 6.2 <https://grafikart.fr/formations/react>
- 6.3 Documentation for all available endpoints in the todolist RESTful API : <https://swagger.todos.in.jt-lab.ch/>
- 6.4 Documentation to make specific queries to the RESTful API : https://docs.postgrest.org/en/v12/references/api/tables_views.html
- 6.5 <https://github.com/pmndrs/zustand>
- 6.6 Reminders for git usage: <https://www.figma.com/board/NZjAxUSIE0EiSv9uavCoKK/Utilisation-de-Git?node-id=0-1&t=vDTVZQMDjoi1CHqi-1>

7 Evaluation criteria

- 7.1 All the tasks from the GitHub project have been implemented. For each task there is a pull request, with a reviewer, an estimate, a final time, and a **close keyword**.
- 7.2 All the commits must comply with the [conventional commits](#) standard.
- 7.3 The codebase must be formatted and linted with eslint and prettier (no format errors, no lint errors, no type errors).
- 7.4 The app deployed to GitHub pages must be working.
- 7.5 The app build process must work out of the box `pnpm build`.
- 7.6 Your codebase respects the react team recommendations (see links on each project task).
- 7.7 The project respects TypeScript and web best practices.
- 7.8 The README contains a brief description of the app, a list of features, a diagram that shows each react components and their relationships, and cli instructions to launch the app.
- 7.9 The style of the app is clean and modern. Use recent CSS features.
- 7.10 All features work as described in the project tasks.
- 7.11 There is no dead code.

